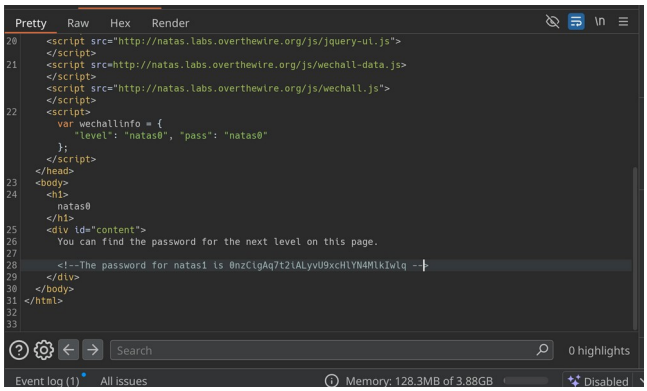
Natas 0

Looking at the source code, I noticed the hint that the password was “on this page” suggesting that I check for hidden text. Looking at the page source, I found the flag in a HTML comment.



Flag: scfWG6qNEIdzqVyfRwEGXyNUfFZkZeQ7
Vulnerability: (CWE-312) Cleartext Storage of Sensitive Information
Impact: Sensitive credentials exposed
Mitigation: Avoid storing sensitive data in client side code. Encode it if absolutely necessary

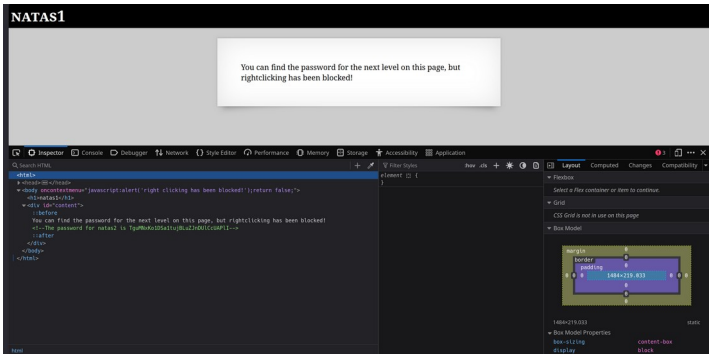
Natas 0→1

I needed to find the flag again without being able to inspect element, as javascript was preventing me from right clicking the website.

To see the html code, there are two methods.

Method 1: Disabling Javascript
I disabled javascript in the browser with a browser extension so the page couldn’t run client-side code. After reloading the page, I could inspect element again and found the flag as a comment in the HTML code.

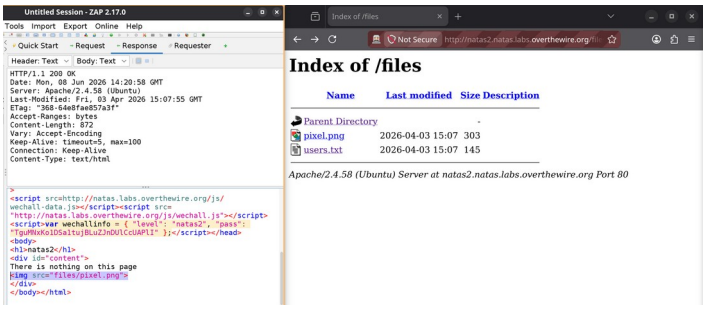
Method 2: Using an intercepting proxy
I used OWASP’s Zaproxy to load the page. This allowed me to view the HTML response without being blocked by javascript, showing me the flag as a comment in the HTML code.



Flag: vsDOxoXyq3wckCP1ZmTZ71ngIA606odB
Vulnerability: (CWE-312) Cleartext Storage of Sensitive Information
Impact: Sensitive credentials exposed
Mitigation: Avoid storing sensitive data in client side code. Encode it if absolutely necessary

Natas 1→2

When I opened this page, I immediately went to the html to see if there was a hidden flag there. However there wasn't. Instead there was a new line of html that displayed an image called "files/pixel.png" that was exactly 1 pixel big that wasn't in any of the previous levels. I then tried adding "/files/" to the original url to see what was in that directory, such that the new url was "http://natas2.natas.labs.overthewire.org/files/"



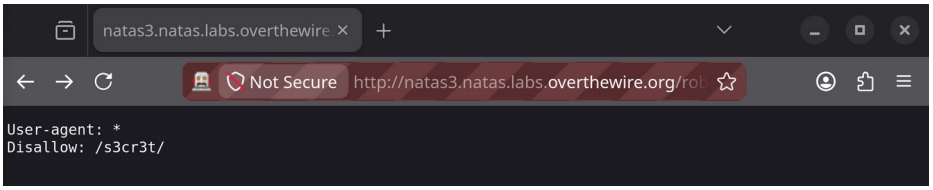
This showed me all the files in the directory of the server, which contained the pixel.png, but also another file called users.txt. Clicking on the link, you can find the flag as the file contains the usernames and passwords for several accounts.

Flag: K30JrSRHzjxq3paUQuwozY4MNvmNFyhI
Vulnerability: (CWE-548) Exposure of information through directory listing
Impact: Credential information exposed
Mitigation: Disable directory listing. Restrict access to sensitive files
Rabbit Hole: If a website owner decides to not hide these files, then they are relying on security through obfuscation. This however can be defeated by Google dorking, which is when you use special search operators whilst searching to find urls/paths that aren't normally visible – so for example, to find this /files directory, we could just search up "inurl:/files", and it would just show up like a normal google search. This however is potentially dangerous for the attacker, as many honeypots are set up to be 'vulnerable' to Google dorking.

Natas 2→3

Similar to the last level, there is nothing on the page, however this time the 1 pixel image had been removed and been replaced with a comment saying not even Google will find it.

Google finds websites to put in their search engine by using a web crawler. Often, websites will put information in a file called robots.txt to ask web crawlers not to visit certain pages. Whilst not enforceable, 'polite' crawling means you follow the website owner's will and 'not find' the page. The clue suggests that we should visit the robots.txt document.



In the url "http://natas3.natas.labs.overthewire.org/robots.txt", a hidden directory was revealed called "/s3cr3t/". In this directory is another user.txt file that contains the flag.

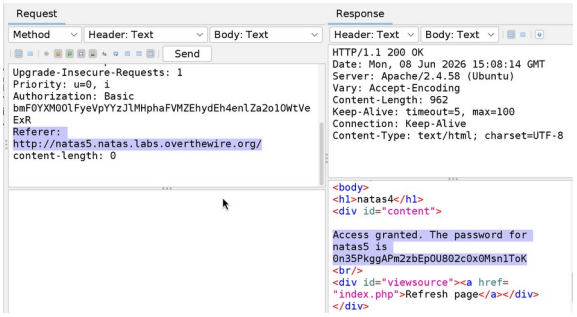
Flag: JDrPnuZAKyl6MkiqQGFIddrqpvgOASth
Vulnerability: (CWE-548) Exposure of information through directory listing
Impact: Credential information exposed
Mitigation: Don't list sensitive paths in robots.txt. Restrict access to sensitive files
Rabbit Hole 1: Whilst the robots.txt document can request that certain bots (which are defined by their user agents) not access certain pages, the document is also used to ask web crawlers to crawl at a certain rate! This is because some website owners could not cope with the amount of requests a very fast bot would put on their server. Not only that, but the document can keep track of
Rabbit Hole 2: Typically our browser attaches a user agent that said the architecture of your computer, the browser version, and on what operating system so the website can return the correct version of a page. With this information, websites can block say, everyone on Linux. The way around this however is user-agent spoofing, where you replace your user agent to say that you are Windows! This will work most of the time, but can sometimes defeated by fingerprinting technique, such as seeing if your computer has the Adwaita font installed (Linux, Gnome only font).

Natas 3→4

The challenge of this level is that you are only allowed to see the flag if you visited the level from level 5's page. Obviously you can't access level 5 without getting the flag from level 4, so we are stuck.

To solve this level, we first must understand how websites even know which page you are coming from. There are many ways to do this, for example, tracking requests made to your site and cross referencing those requests with the user's session cookie to build a timeline of which pages they visited. Another way to do this is by setting a referrer header that contains the url of the previous website you were on.

Looking at the requests made from previous challenges, we can see that overTheWire's Natas uses the referer header instead of tracking cookies. This means that if we try using this header to spoof coming from natas5, we can get the flag!



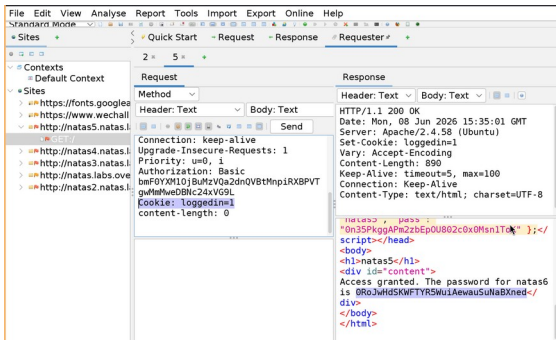
Flag: 0n35PkggAPm2zbEpOU802c0x0Msn1ToK
Vulnerability: (CWE-346) Origin Validation Error

Impact: Unauthorised access to restricted resources through spoofing of client-controlled headers
Mitigation: Don't rely on client-controlled headers for authorisation, instead using secure tokens for sessions
Rabbit Hole: Whilst there are legitimate reasons to have a referrer header, this could be an information leak/security risk that would harm the user eg: You click on an insurance website from a cancer website – the insurance company now knows you likely have, or are related to someone who is suffering from cancer and can charge you higher premiums for medical insurance, and all you did was click a link. For this reason, modern browsers offer a no-referrer policy, in that they strip any referrer information from a request and replace it with "no-referrer".

Natas 4→5

The challenge of this level is that you have to be logged in to view the flag. I started by looking at the request made to the site and comparing it to previous requests I have made to the previous levels. I noticed that the request made to this level featured this header: "Set-Cookie: loggedin=0:", which wasn't present in any of my previous requests.

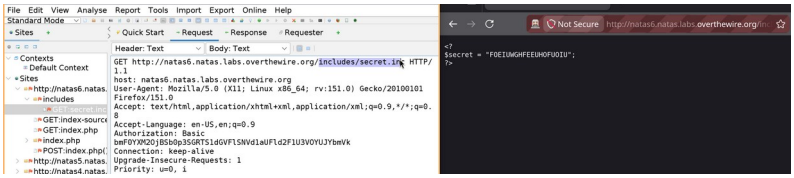
The vulnerability here is that being logged in is controlled by an attacker-controlled cookie! Making a request where loggedin=1, we gain access to another page which contains the flag.



Flag: 0RoJwHdSKWFTYR5WuiAewauSuNaBXned
Vulnerability: (CWE-784) Reliance on Cookies without Validation and Integrity Checking in a Security Decision
Impact: Unauthorized access through client-side session control
Mitigation: Use a more sophisticated, harder to forge, high entropy session token management system. Validate session tokens in the server for every request, and ensure that sesion tokens have a short expiration time
Rabbit Hole: Whilst a cookie containing loggedin seems like an absurd vulnerability, it turns out that this isn't too far from industry implementation. However instead of a boolean, websites like Instagram will provide a very long, high entropy session token to authenticate you. So it is still possible that you enumerate to find other people's session tokens, but the computational power needed is unfeasible. Not only that, but with websites that contain very sensitive information, like Instagram or any banking website, even if you find/steal someone's valid session cookie, you may be asked to re authenticate yourself as you are now connecting from a different IP!

Natas 5→6

Starting this challenge, I assumed that getting the flag would come from an SQL injection. However, when I looked at the source code offered, I saw that the database lookup is done as an array lookup in php, and that the flag for natas7 is stored in this database. It however, is only revealed if you input a secret key. This key however is loaded from a file "/includes/secret.inc".



If we look at the file, we can see the ‘secret’ the php code is loading, and input that to get our flag.

Flag: bmg8SvU1LizuWjx3y7xkNERkHxGre0GS
Vulnerability: (CWE-548) Exposure of information through directory listing
Impact: Secret key exposed through web-accessible directories
Mitigation: Use proper file and directory permissions to prevent public access to sensitive files, and disable directory listing.
Rabbit Hole: JavaScript and PHP are both scripting languages. The main difference however is that PHP is server side, whilst JS is browser side. This makes PHP inherently more secure when handling sensitive information by being able to control both the request and the data on a trusted server. Whereas since JavaScript is client side, the requests and data would be viewable to the attacker. Not only that, but JS requests can be controlled by the attacker!

Natas 6→7

At first I immediately saw the hint saying the password was in “/etc/natas_webpass/natas8”, so I concatenated that to the url and for a 404 error. Since the /etc/ folder is only accessible with privilages in linux, I thought I may need to make myself appear as an admin, so I researched the Authorization header, to see other options besides “Basic”. Going down this rabbit hole, I discovered .htpasswd and .htaccess for Apache servers from “https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Authentication#authentication_schemes”. Since the Natas server is Apache, I tried visiting these resources, which both gave me a 401 error. This was a complete dead end.

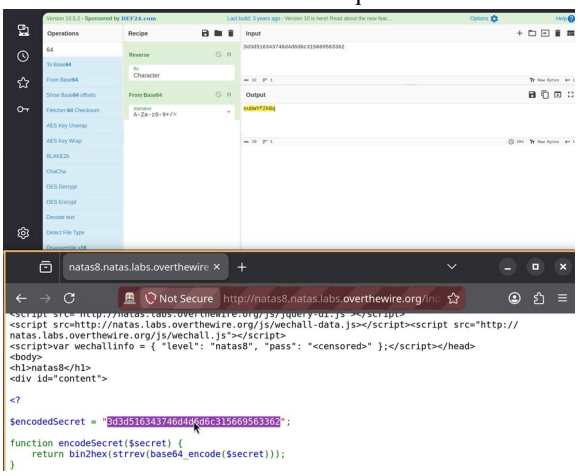
I then thought about how different the url for “Home” and “About” was compared to what I had previously seen. Beforehand, accessing home would be concatenating “/Home” to the url, but now it was concatenating “?page=home”. So I tried accessing the password page this way with “http://natas7.natas.labs.overthewire.org/index.php/includes/secret.inc/index.php?page=/etc/natas_webpass/natas8”, which revealed the flag.

Flag: xcoXLmzMkoIP9D7hlgPlh9XD7OgLAe5Q
Vulnerability: (CWE-22) Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
Impact: Arbitrary file read access, exposing sensitive files
Mitigation: Validate and sanitize user input. Remove read permissions for files the program shouldn’t see
Rabbit Hole: .htaccess and .htpassword are two files on Apache servers that allow access control without PHP. This is stronger/more secure than checking through PHP as checking is now automated by the server and avoids the need to write any complex authentication code, or storing passwords in a separate file that requires more effort and precision to set up securly. The disadvantages however is that the server no longer has complex authentication eg: ban this IP after 7 failed attempts at a password in the space of 10 seconds. Ideally, access would be controlled by PHP, and passwords by .htpassword.

Natas 7→8

Like Natas 6→7, we once again have a PHP database that requires a particular key to reveal the flag, however this time, they have encrypted the key.

The vulnerability here is that not only that I can view the encryption methods, all the methods of encryption being used, are not one way functions, nor is a key is necessary to decrypt the cipher. Because of this, we can reverse the resulting ciphertext to get the key: Translate from hex to binary → reverse the string → decode from base64, giving the key “oubWYf2kBq”.

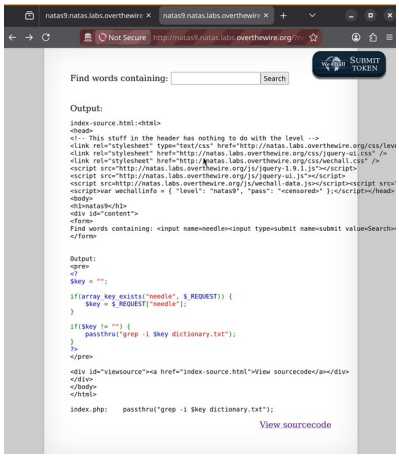


Putting this into the query, we get the flag back.

Flag: ZE1ck82lmdGIoErlhQgWND6j2Wzz6b6t
Vulnerability: (CWE-327) Use of a Broken or Risky Cryptographic Algorithm
Impact: Encryption keys are easily decrypted, making the encryption useless and just security theatre
Mitigation: Use industry standard cryptography for sensitive data for example: bcrypt, AES and SHA. Also don’t rely on cryptography alone for security.

Natas 8→9

I started this by looking at the source code, and the urls when I queried the database. Interestingly, anything I input, apart from “”, seems to be put directly into the grep command. So instead of grepping for a word, I went to the man page to see if there are any useful flags to input that could spit out the entire file. I tried “-r” and it spat out the source code.



I have no idea why it did this since I don’t know PHP, but I assume then, that it is a result of an error. This made me think that it was prone to an injection attack. So I tried the payload “a; cat” to see if it would run “cat dictionary.txt” and that

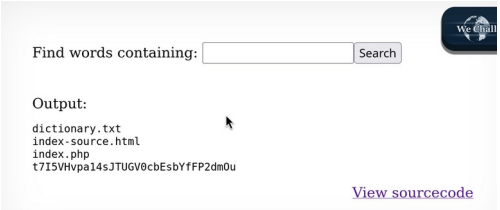
worked.The issue now however is that dictionary.txt has too many lines to manually inspect so I tried looking for words larger than 25 characters long with “a; grep -oE '.{25,}'” , as I think the flag will be around that long. This outputted no results 😕

Seeing as the password was probably not in the dictionary.txt file, I then decided to look laterally around the directory with “a; ls -a #” which revealed that the .htaccess and .htpasswd files were there.

Running the injection “a; cat .htpasswd #” revealed a password for natas9, so I decided to explore the parent directory with “; ls -a ../ #”, to see what else I could access.

Seeing the other directories, I tried seeing the contents of natas10, which was blank. After exploring the file system for a little bit, I tried “; cat ../../html/index.html #”, which actually loaded me onto the base natas page.

I was a little lost until I remembered the password folder revealed in level 7 and ran “; cat /etc/natas_webpass/natas10 #”, showing the flag.



Flag: t7I5VHvpa14sJTUGV0cbEsbYfFP2dmOu
Vulnerability: (CWE-78) Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Impact: Remote code execution
Mitigation: Never pass user input directly to the shell, use parameterised and pre-prepared commands. Strict santisation and validation for all user input

Natas 9→10

For this level, it is obvious the goal is to do the same method as above, but while getting around the sanitizer (preventing the symbols ‘|’, ‘;’ and ‘&’). This means that any form of sequential operations is not possible.

I then surfed the web for any way to replicate this behaviour without those characters and found nothing. So instead I thought of ways I could use the grep to my advantage and realised that I could grep for the flag in /etc/natas_webpass/natas11! With this payload “a /etc/natas_webpass/natas11 #”, the flag was revealed. If ‘a’ wasn’t a character in the flag, I could just keep trying other letters until it got revealed.

Flag: UJdqkK1pTu6VLt9UHWAgRZz6sVUZ3lEk
Vulnerability: (CWE-78) Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Impact: Remote code execution
Mitigation: Never pass user input directly to the shell, use parameterised and pre-prepared commands. Strict santisation and validation for all user input

Natas 11→12

Looking through the source code, it looks like the php strips off the extension, replaces it with .jpg and stores it in the server all on client side. This seems vulnerable as I can intercept the request and just change the extension back.

I created a php file to dump the contents of '/etc/natas_webpass/natas13' to place onto the server. Then I intercepted the request and changed the extension from .jpg back to .php. This was the file

"<?php
echo file_get_contents('/etc/natas_webpass/natas13')
?>"

Then I visited the file in the server, which contained only the flag in a html body

Flag: trbs5pCjCrkuSknBBKHhaBxq6Wm1j3LC
Vulnerability: (CWE-434) Unrestricted Upload of File with Dangerous Type
Impact: Remote Code Execution
Mitigation: Validate file types on server side as opposed to client side. Disable execution privileges for images/user provided files

Natas 12→13

The patch that this level has done for the previous level is that it checks the image is a real image. It does this using the PHP function exif_imagetype(), which looks at the magic bytes and ONLY the magic bytes (which is a vulnerability)

So the plan is to make a php file similar to before, but to fake the magic bytes, then intercept the request again to return our file back to it's .php extension.
Doing this gave us the output of ���z3UYcr4v4uBpeX8f7EZbMHlzK4UR2XtQ, which is mainly the flag.

Flag: z3UYcr4v4uBpeX8f7EZbMHlzK4UR2XtQ
Vulnerability: (CWE-434) Unrestricted Upload of File with Dangerous Type
Impact: Remove Code Execution
Mitigation: Validate file types using harder to spoof methods. Disable execution privileges for images/user provided files

Natas 13→14

Looking at the source code, we can see that for this challenge, the site is using a sqlite database to store login credentials. We get the flag by logging in as anyone.

Since the user input is not santisized, nor is there a seperation of data and code in the query, this database is vulnerable to a sql injection atta ck. Inputing a payload of 'or '1=1' – is enough to log us in as anyone, giving us the flag.

Flag: SdqIqBsFcz3yotlNYErZSZwblkm0lrvx
Vulnerability: (CWE-89) Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Impact: Authentication bypass, unauthorized database access
Mitigation: Use parameterized queries/pre-prepared queries. Implement input validation and sanitisation of user input

Natas 14→15

Unlike the previous level, the server no longer echos back the query results, but rather 3 predefined sentences. However, the previous vulnerability remains unpatched. To figure out the flag, we must do a blind sql attack in which we need to figure out every character individually, by asking the server if the character at that place is correct, assuming that we are finding the password for Natas16.

Since the password is very long, it helps to automate this enumeration with a script. I used one that I found for this challenge at: https://github.com/EkRafz/OverTheWire---Walkthroughs/blob/main/Natas/Walkthroughs/natas-level-14-level-15.md

Flag: hPkjKYviLQctEW33QmuXL6eDVfMW4sGo
Vulnerability: (CWE-89) Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Impact: Blind SQL injection allowing data exfiltration leading to authentication bypass
Mitigation: Use parameterized queries/pre-prepared queries. Implement input validation and sanitisation of user input

At this point, the wargame wasn't very fun anymore and I realised my interest wasn't in web, but rather more low level vulnerabilities. That, combined with the fact that they switched out the flags after I finished this level (so I would have to do it all again), demotivated me to finish the wargame, and instead I went and explored my interest in binary exploitation. Sorry if you really wanted the rest of the levels, but I have no intent to finish this :(

Natas: Final Takeaways and Summary

This wargame helped me practice web vulnerabilities. I realized whilst doing the challenges, that web vulnerabilities are caused mainly by obvious improper configurations, as opposed to becomes vulnerable because of a 0day that was too hard for the defenders to spot. Another thing I realized was that, with all these challenges, user-side validation/sanitation, as well actually using user input is a huge attack surface. To mitigate this, it is probably better to use multi-choice buttons as pre-prepared options for the user instead of actually taking in their input. I also learnt the importance of being flexible. A lot of these challenges aren't solved with specialist knowledge, the way binary exploitation requires an in depth understanding of the operating system and compiler, but rather general problem solving combined with surface level skills, such as figuring out you must change the file extension after your browser has 'sanitized' the file you gave it.